

Clean CSS Architecture

How to organize your stylesheets so they stay easy to maintain as the project grows. No frameworks, just CSS that already works in every current browser.

1 Order the cascade with @layer

Declare the layer order once, at the top. A later layer beats an earlier one regardless of selector specificity. Styles left outside every layer beat all of them.

utilities	.p-0, .text-center
components	.card, .btn, .nav
layout	grid, containers
base	html, body, h1, a
reset	clears browser defaults

Bottom, lower priority · Top, higher priority

```
@layer reset, base, layout, components, utilities;

@layer components {
  .btn { padding: var(--space-2) var(--space-4); }
}

@layer utilities {
  .p-0 { padding: 0; } /* beats .btn */
}
```

3 Class names with BEM

Block, element, modifier. The name tells you which component each class belongs to, and the selector stays flat: one class, the same specificity across the whole project.

CLASS	WHAT IT IS
.card	Block: standalone component
.card__title	Element: part of the block
.card--featured	Modifier: a variant of the block

AVOID

```
.card .content h3 { }
```

BETTER

```
.card__title { }
```

2 Design tokens in :root

Define colors, spacing, radii and type as custom properties. When the client changes the brand, you edit one place.

AVOID

```
.header { background: #14213d; }
.footer { background: #14213d; }
.btn { background: #14213d; }
```

BETTER

```
:root {
  --color-primary: #14213d;
  --space-4: 1rem;
  --radius: 0.5rem;
}

.header,
.footer,
.btn { background: var(--color-primary); }
```

4 Keep specificity under control

The lower and more even your specificity, the fewer rules override each other. Read it as (IDs, classes, tags).

SELECTOR	WEIGHT	
#menu .link	1,1,0	Don't style with IDs
.card .content h3	0,2,1	Tied to the HTML
.card__title	0,1,0	The target
:where(.nav) a	0,0,1	:where() weighs 0; good in base

Keep !important for the utilities layer only, where the rule must always win.

5 Fluid sizes with clamp()

One value that scales between a minimum and a maximum based on screen width. It replaces several media queries for type and spacing.

```
:root {
  --text-base:
    clamp(1rem, 0.95rem + 0.25vw, 1.125rem);
  --text-h1:
    clamp(2rem, 1.5rem + 2.5vw, 3.5rem);
}

h1 { font-size: var(--text-h1); }
```

clamp(minimum, preferred value, maximum). Use rem for the min and max so the user's zoom still works.

6 Responsive grid without media queries

auto-fit creates as many columns as will fit. min() keeps the card from overflowing on narrow screens. The container sets the space between items with gap.

```
.grid {
  display: grid;
  grid-template-columns:
    repeat(auto-fit, minmax(min(16rem, 100%), 1fr));
  gap: var(--space-4);
}
```

AVOID

```
.card { margin: 0 16px 16px 0; }
```

BETTER

```
.grid { gap: var(--space-4); }
```

7 Dark mode by swapping tokens

If components use variables, dark mode means redefining those variables. You don't touch a single component.

```
:root {
  --color-bg: #ffffff;
  --color-text: #1a1a1a;
}

@media (prefers-color-scheme: dark) {
  :root {
    --color-bg: #121212;
    --color-text: #eaeaea;
  }
}

body {
  background: var(--color-bg);
  color: var(--color-text);
}
```

8 Review before you commit

- ☐ Is there a hard-coded color or size that should be a token?
- ☐ Does any selector depend on HTML structure, like `.card .content h3`?
- ☐ Did you use an ID or `!important` outside the utilities layer?
- ☐ Is every rule in the layer it belongs to?
- ☐ Does spacing between items come from `gap` or a spacing token?
- ☐ Does the page look right between 320 px and 1440 px wide?

9 One file per responsibility

Each file has one purpose and enters its layer from main.css. Tokens stay outside the layers because they only declare variables.

```
css/
├─ main.css
├─ tokens.css
├─ reset.css
├─ base.css
├─ layout.css
├─ components/
│  ├─ button.css
│  └─ card.css
└─ utilities.css

/* main.css */
@layer reset, base, layout, components, utilities;
@import url("tokens.css");
@import url("reset.css") layer(reset);
@import url("base.css") layer(base);
@import url("layout.css") layer(layout);
@import url("components/button.css") layer(components);
@import url("components/card.css") layer(components);
@import url("utilities.css") layer(utilities);
```

@import comes before any other rule; only @charset and @layer may precede it. In production, bundle the files with Vite or Lightning CSS.